



INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Linguagem de programação C++

Estrutura de dados heterogêneas

Registro

Um registro é um mecanismo da linguagem C para agrupar diversas variáveis, que inclusive podem ser de tipos diferentes, mas que, dentro de um contexto, fazem sentido serem tratadas conjuntamente.

Exemplos de usos de registros:

-) Registro de alunos para guardar dados como nome, RA, médias de provas e médias de laboratórios.
-) Registro de pacientes para guardar dados como nome, endereço e histórico de doenças.
-) Registro de clientes para guardar dados como nome, endereço, CPF, telefone e número do cartão de crédito.

Definir registro

Para definirmos um tipo de registro, usamos a palavra reservada struct da seguinte forma:

```
struct nome_do_tipo {  
    tipo_1 nome_do_campo_1;  
    tipo_2 nome_do_campo_2;  
    ...  
    tipo_n nome_do_campo_n;  
};
```

Exemplo:

```
struct Aluno {  
    string nome;;  
    int idade;  
    char sexo;  
};
```

```
int main() {  
    /* Declarando variáveis registros do tipo Aluno */  
    struct Aluno a, b;  
    ...  
    return 0;  
}
```

Usando uma variável tipo registro

- Podemos acessar individualmente os campos de uma determinada variável registro como se fossem variáveis comuns.
- Neste caso, a sintaxe é a seguinte: `variavel_registro.nome_do_campo`
- Os campos individuais de um variável registro têm o mesmo comportamento de qualquer variável do tipo do campo.

```
#include <stdio.h> int main() {
#include <string.h> struct Aluno a, b;
                        a.nome="Helen";
struct Aluno {          a.idade = 18;
string nome;           a.sexo = 'F';
int  idade;            cout<<" Digite nome, idade e sexo(f/m):";
char sexo;             getline(cin,b.nome);
};                     cin>>b.idade >> b.sexo;
                        cout<<"nome A:"<<a.nome<<"Idade a:"<< a.idade<< "Sexo a:"<< a.sexo;
                        cout<<"nome B:"<<b.nome<<"Idade : "b<< b.idade<< "Sexo b:"<< b.sexo;
                        return 0;
}
```

Atribuição de registros

Podemos atribuir (copiar) um registro para outro diretamente:

```
variavel_registro_1 = variavel_registro_2;
```

Neste caso é feita uma cópia de todos os campos do registro variável registro 2 para o registro variável registro 1.

```
int main() {  
    struct Aluno a, b;  
    cout<<" Digite nome, idade e sexo(f/m) :";  
    getline(cin,b.nome);  
    cin>>b.idade >> b.sexo;  
    a=b;  
    cout<<"nome A:"<<a.nome<<"Idade a:"<< a.idade<< "Sexo a:"<< a.sexo;  
    cout<<"nome B:"<<b.nome<<"Idade : "b<< b.idade<< "Sexo b:"<< b.sexo;  
    return 0;  
}
```

Vetor de registro

Podemos declarar um vetor de registros quando necessitamos de várias instâncias de um mesmo tipo de registro, por exemplo, para cadastrar todos os alunos de uma mesma turma.

- Como declarar um vetor de registros: `struct tipo_do_registro nome_do_vetor[MAX];`
- Como acessar um campo de registro de um vetor: `nome_do_vetor[indice].campo`

```
#include <stdio.h>
struct Aluno {
    string nome;
    double nota;
};

int main () {
    struct Aluno turma[10];
    int i;
    double nota = 0.0;

    for (i = 0; i < 10; i++) {
        cout<<"Digite o nome do aluno: ";
        getline(cin,turma[i].nome);
        cout<<"Digite a nota do aluno: ";
        cin>>turma[i].nota);
    }
    for (i = 0; i < 10; i++)
        nota = nota + turma[i].nota;
    cout<<"Media da turma:" << nota / 10);

    return 0;
```

Redefinindo um tipo

- Para facilitar a compreensão e melhorar a organização dos códigos, podemos declarar um tipo próprio, que possui a mesma função de outro tipo já existente.
- Por exemplo, em um programa onde manipulamos médias de alunos, podemos declarar as variáveis relacionadas com o campo nota como nota e não double.

A forma de se fazer isso é utilizando o comando typedef, seguindo a sintaxe:

```
typedef tipo_existente tipo_novo;
```

- Usualmente, fazemos essa declaração fora da função main(), embora também seja permitido declarar um novo tipo dentro de uma função.

Exemplo: typedef double nota;

- O exemplo acima cria um novo tipo, chamado nota, cujas variáveis desse tipo serão pontos flutuantes (double).

- Assim como podemos redefinir o tipo struct Aluno como simplesmente Aluno:

```
typedef struct Aluno Aluno;
```

- O comando typedef pode ser usado junto com a definição de struct, determinando o nome para o novo tipo.

```
struct Aluno {
```

```
    int ra;
```

```
    double nota;
```

```
};
```

```
typedef struct Aluno Aluno;
```

```
//ou
```

```
typedef struct {
```

```
    int ra;
```

```
    double nota;
```

```
} Aluno;
```


Exemplo: controle estoque

- preencha o tipo, descrição, preço e a quantidade de cada produto;
- mostrar o vetor de produtos e o tipo;
- buscar por tipo e mostrar produtos desse tipo;

```
#include <iostream>
using namespace std;
#define MAX 5
typedef struct{
    int tipo;
    string descricao;
    float preco;
    int quantidade;
}Produto;
```

```
int main()
{
    Produto estoque[MAX];
    int tipo;
    for(int i=0;i<MAX; i++){
        cout<<"Digite a descricao do produto:";
        cin>>ws;//limpar buffer;
        getline(cin, estoque[i].descricao);
        cout<<"Digite tipo, preço, quantidade em estoque";
        cin>>estoque[i].tipo>>estoque[i].preco>>estoque[i].quantidade;
    }
    cout<< "Produtos e tipo:";
    for(int i=0; i<MAX; i++){
        cout<<"Produto: "<<estoque[i].descricao<< "tipo:"<<estoque[i].tipo<<endl;
    }
    cout<<"Digite o tipo buscado: 1- alimento 2- limpeza";
    cin>>tipo;
    for(int i=0; i<MAX; i++){
        if(tipo==estoque[i].tipo)
            cout<<"Produto:"<<estoque[i].descricao<<endl;
    }

    return 0;
```